

练习 6.4.3: 使用图 6-22 所示的翻译方案来翻译下列赋值语句:
2) $x = a[i][j] + b[i][j]$

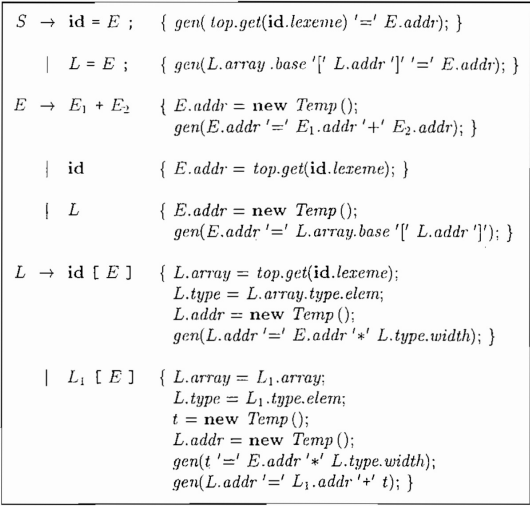


图 6-22 处理数组引用的语义动作

$t_1 = i * row_size_a$
 $t_2 = j * ele_size_a$
 $t_3 = t_1 + t_2$
 $t_4 = a[t_3]$
 $t_5 = i * row_size_b$
 $t_6 = j * ele_size_b$
 $t_7 = t_5 + t_6$
 $t_8 = b[t_7]$
 $t_9 = t_4 + t_8$
 $x = t_9$

练习 6.4.6: 一个按行存放的整数数组 $A[i, j]$ 的下标 i 的范围为 1~10, 下标 j 的范围为 1~20。每个整数占 4 个字节。假设数组 A 从 0 字节 开始存放, 请给出下列元素的位置:

2) $A[10, 8]$

$9 \times 20 \times 4 + 7 \times 4 = 748$

练习 6.4.8: 一个按行存放的实数型数组 $A[i, j, k]$ 的下标 i 的范围为 1~4, 下标 j 的范围为 0~4, 且下标 k 的范围为 5~10。每个实数占 8 个字节。假设数组 A 从 0 字节开始存放。计算下列元素的位置。

2) $A[1, 2, 7]$

$(0 \times 5 \times 8 + 2 \times 8 + 7) \times 8 = 112$

练习 6.5.2: 像 Ada 中那样, 我们假设每个表达式必须具有唯一的类型, 但是我们根据一个子表达式本身只能推导出一个可能类型的集合。也就是说, 将函数 E_1 应用于参数 E_2 (其文法产生式为 $E \rightarrow E_1(E_2)$) 有如下规则:

$E.type = \{ \textcircled{0} \}$ 对 $E_2.type$ 中的某个 s , $s \text{---} \textcircled{0}$ 在 $E_1.type$ 中 |
描述一个可以确定每个子表达式的唯一类型的语法制导定义(SDD)。它首先使用属性 $type$, 按照自底向上的方式综合得到一个可能类型的集合。在确定了整个表达式的唯一类型之后, 自顶向下地确定属性 unique 的值, 这个属性表示各个子表达式的类型。

- 本题文法: $S \rightarrow id = E \mid E \rightarrow E_1 + E_2 \mid -E_1 \mid (E_1) \mid id [E_1(E_2)$

① 自底向上

$E \rightarrow id = E, E.type = \{ symtab[id].type \}$

$E \rightarrow E_1 + E_2, E.type = \{ t \mid \exists t_1 \in E_1.type \wedge \exists t_2 \in E_2.type \wedge t \text{ 在 } (t_1, t_2) \rightarrow t \text{ 上有定义} \}$

$E \rightarrow -E_1, E.type = E_1.type$

$E \rightarrow (E_1), E.type = E_1.type$

$E \rightarrow id, E.type = id.type$

$E \rightarrow E_1(E_2) E.type = \{ t \mid \exists s \in E_2.type \wedge E_1 \text{ 在 } s \rightarrow t \text{ 上有定义} \}.$

② 自顶向下

产生式	语义规则
$S \rightarrow id = E$	$E.unique = id.type \text{ if }$ $id.type \in E.type \text{ else error()}$
$E \rightarrow E_1 + E_2$	$E_1.unique = E.unique$ $E_2.unique = E.unique$
$E \rightarrow -E_1$	$E_1.unique = E.unique$
$E \rightarrow (E_1)$	$E_1.unique = E.unique$
$E \rightarrow id$	
$E \rightarrow E_1(E_2)$	$E_2.unique = s \text{ if }$ $s \in E_2.type \wedge E_1 \text{ 在 } s \rightarrow t \text{ 上有定义}$ $\text{else error()}. $ $E_1.unique = t \text{ if } \dots (\text{同上})$

将①②两步结果合并即可。

练习 6.6.1: 在图 6-36 的语法制导定义中添加处理下

列控制流构造的规则:

- 1) 一个 repeat 语句, **repeat** S **while** B 。
- ! 2) 一个 for 循环语句, **for** (S_1 ; B ; S_2) S_3 。

产生式	语义规则
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \parallel label(S.next)$
$S \rightarrow assign$	$S.code = assign.code$
$S \rightarrow if(B) S_1$	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \parallel label(B.true) \parallel S_1.code$
$S \rightarrow if(B) S_1 else S_2$	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\parallel label(B.true) \parallel S_1.code$ $\parallel gen('goto' S.next)$ $\parallel label(B.false) \parallel S_2.code$
$S \rightarrow while(B) S_1$	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \parallel B.code$ $\parallel label(B.true) \parallel S_1.code$ $\parallel gen('goto' begin)$
$S \rightarrow S_1 S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \parallel label(S_1.next) \parallel S_2.code$

图 6-36 控制流语句的语法制导定义

1) 产生式

语义规则

$S \rightarrow repeat S_1 while B$

$S_1.next = new label()$
 $B.true = new label()$
 $B.false = S.next$
 $S.code = label(B.true) \parallel$
 $S_1.code \parallel$
 $label(S_1.next) \parallel$
 $B.code$

2) 产生式

语义规则

$S \rightarrow for(S_1; B; S_2) S_3$

$S_1.next = new label()$
 $B.true = new label()$
 $B.false = S.next$
 $S_2.next = S_1.next$
 $S_3.next = new label()$
 $S.code = S_1.code \parallel label(S_1.next)$
 $\parallel B.code \parallel label(B.true)$
 $\parallel S_2.code \parallel label(S_2.next)$
 $\parallel S_3.code \parallel gen('goto',$
 $S_1.next)$

练习 6.7.3: 当使用图 6-46 中的翻译方案对图 6-47 进行翻译时, 我们为每条语句创建 $S.next$ 列表。--开始是赋值语句 S_1, S_2, S_3 , 然后逐步处理越来越大的 if 语句、if-else 语句、while 语句和语句块。在图 6-47 中有 5 个这种类型的结构语句:

- S_4 : **while** (E_3) S_1 。
 S_5 : **if** (E_4) S_2 。
 S_6 : 包含 S_5 和 S_3 的语句块。
 S_7 : 语句 **if** (E_2) S_4 **else** S_6 。
 S_8 : 整个程序。

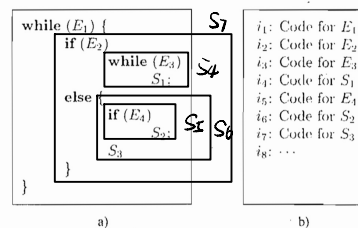


图 6-47 练习 6.7.2 的程序的控流结构

对于这些结构语句, 我们可以通过一个规则用其他的 $S_j.next$ 列表以及程序中的表达式的列表 $E_k.true$ 和 $E_k.false$ 构造出 $S_i.next$ 。给出计算下列 $next$ 列表的规则:

- (1) $S_4.next$ (2) $S_5.next$ (3) $S_6.next$ (4) $S_7.next$ (5) $S_8.next$

(题目有误, 这里应该使用 nextlist, truelist 和 falselist)

(1) $S_4.nextlist = E_3.falselist$

(2) $S_5.nextlist = E_4.falselist \cup S_1.nextlist$

(3) $S_6.nextlist = S_2.nextlist$

(4) $S_7.nextlist = S_4.nextlist \cup S_6.nextlist$

(5) $S_8.nextlist = E_1.falselist$